FASTEIG User's Guide

Based on the FASTEIG source distribution

Repository version 1.0.0
Guide revised August 7, 2026

Contents

1 Introduction **2**

2 Prerequisites **2**
2.1 Required software . . . 2
2.2 Optional CPU packages . . . 2
2.3 Optional GPU packages . . . 3

3 Building with CMake **3**
3.1 Basic CPU build . . . 3
3.2 CMake build with Intel oneMKL . . . 3
3.3 CMake build with Cray LibSci . . . 4
3.4 Current CMake limitations . . . 4

4 Building the Command-Line Programs **4**
4.1 CPU executable . . . 4
4.2 GPU executable . . . 5
4.3 Installing and enabling cuDSS . . . 5

5 Preparing Matrices and an Input File **5**
5.1 Matrix requirements and formats . . . 5
5.2 Input-deck syntax . . . 6
5.3 Input keys . . . 7
5.4 Choosing a solver and target . . . 8

6 Running on CPUs **8**

7 Running on NVIDIA GPUs **8**

8 Verification **9**

9 Troubleshooting **9**

10 Quick Reference **10**

1 Introduction

FASTEIG is a C++17 library and example program for computing selected eigenpairs of large sparse, real matrices. A stored standard problem has the form

$$Ax = \lambda x,$$

and an optional generalized problem has the form

$$Ax = \lambda Bx.$$

The command-line driver reads the matrices and solver settings from a small two-column input file. It can run the supported algorithms on a CPU; CUDA builds can additionally execute Lanczos and Block Lanczos on an NVIDIA GPU.

This guide describes the repository as it currently exists. In particular, Section 3.4 distinguishes the CMake library build from the Makefile-based example and CUDA builds.

2 Prerequisites

2.1 Required software

- A C++17 compiler. The top-level CMake file recognizes GCC, Clang, Intel LLVM (`icpx`), and Intel Classic compilers.
- CMake 3.20 or newer for the CMake library build.
- Eigen headers. Eigen 3.4 is a suitable choice. Eigen is header-only, but its include directory must contain `Eigen/Core`.
- A BLAS/LAPACK implementation. The CMake build can use system BLAS/LAPACK, Intel oneMKL, Cray LibSci, or a fetched OpenBLAS copy.

2.2 Optional CPU packages

Intel oneMKL. Enable oneMKL with `-DUSE_MKL=ON`. FASTEIG deliberately uses the MKL LP64 integer interface and the GNU threading layer. Set `MKLROOT` either in the environment (usually by sourcing the oneAPI setup script) or on the CMake command line. MKL also supplies Pardiso, which FASTEIG uses for CPU shift-and-invert (`which SAI`) when Eigen's MKL integration is enabled.

Cray LibSci. On a Cray system, configure with `-DUSE_LIBSCI=ON` and use the Cray compiler wrappers. The CMake logic looks first for `sci_gnu`, then for the `cray-libsci` pkg-config package. `USE_MKL` and `USE_LIBSCI` are mutually exclusive.

OpenBLAS. With neither MKL nor LibSci selected, CMake searches for system BLAS and LAPACK. If they are unavailable, it fetches OpenBLAS 0.3.27 into `openblas/` in the source tree and builds it. This fallback requires Git and network access during configuration. Installing BLAS/LAPACK through the operating system avoids that download.

2.3 Optional GPU packages

A GPU build requires:

- an NVIDIA GPU supported by the selected CUDA toolkit;
- the CUDA compiler `nvcc`; and
- cuSPARSE, cuSOLVER, cuBLAS, and the CUDA runtime.

NVIDIA cuDSS is optional for ordinary GPU Lanczos calculations, but is required for GPU shift-and-invert calculations (`which SAI`). The repository supplies `tools/install_cudss.sh`, which can install a CUDA 12 or CUDA 13 Python wheel into project space, or unpack an NVIDIA tarball.

3 Building with CMake

Run all commands in this section from the repository root.

3.1 Basic CPU build

If Eigen is already installed, point `EIGEN_DIR` at the directory that directly contains `Eigen/Core`. For example:

```
cmake -S . -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DEIGEN_DIR=/path/to/eigen-3.4.0 \
  -DBUILD_EXAMPLES=OFF \
  -DBUILD_TESTS=OFF
cmake --build build -j 8
```

If `EIGEN_DIR` is omitted, CMake first checks for an existing `eigen/Eigen/Core` below the source directory. Otherwise it fetches Eigen's `master` branch into `eigen/`; this also requires Git and network access.

The resulting static library is `build/lib/libfasteig_Generic.a`. Change its platform suffix with, for example, `-DPLAT=Perlmutter`, which produces `libfasteig_Perlmutter.a`.

3.2 CMake build with Intel oneMKL

After making the oneAPI environment available, use:

```
source /path/to/oneapi/setvars.sh
cmake -S . -B build-mkl \
  -DCMAKE_BUILD_TYPE=Release \
  -DEIGEN_DIR=/path/to/eigen-3.4.0 \
  -DUSE_MKL=ON \
  -DMKLROOT="$MKLROOT" \
  -DBUILD_EXAMPLES=OFF \
  -DBUILD_TESTS=OFF
cmake --build build-mkl -j 8
```

If the environment has not set `MKLROOT`, replace the shell expansion in the command with the actual MKL installation prefix. Do not select the ILP64 interface: the current Eigen/Pardiso path expects LP64 integers.

3.3 CMake build with Cray LibSci

A typical Cray configuration is:

```
CC=cc CXX=CC cmake -S . -B build-libsci \
  -DCMAKE_BUILD_TYPE=Release \
  -DEIGEN_DIR=/path/to/eigen-3.4.0 \
  -DUSE_LIBSCI=ON \
  -DPLAT=Perlmutter \
  -DBUILD_EXAMPLES=OFF \
  -DBUILD_TESTS=OFF
cmake --build build-libsci -j 8
```

The exact environment modules and compiler wrappers are site-specific.

3.4 Current CMake limitations

The current top-level CMake configuration builds the CPU static library only. It does not yet define CUDA sources or a CUDA option. It also does not define an `install()` rule. Therefore "installation" currently means retaining the library in the build tree, or manually copying the archive and the `include/` tree to an application prefix.

Although `BUILD_EXAMPLES` defaults to `ON`, this repository revision does not contain `examples/CMakeLists.txt`; similarly, there is no `tests/CMakeLists.txt`. Use `BUILD_EXAMPLES=OFF` and `BUILD_TESTS=OFF` for a working CMake configuration. Build the supplied command-line programs with the Makefiles as described next.

4 Building the Command-Line Programs

The checked-in Makefiles produce the complete runnable examples and are the current build route for CUDA. They use settings from `make.inc`. That file contains machine-specific paths, so make a local copy and edit it rather than relying on its checked-in values:

```
cp make.inc make.inc.local
# Edit make.inc.local: set CXX, CC, PLAT, EIGEN, MKLROOT,
# CUDA_PATH, CUDA_ARCH, and related flags for this machine.
```

Here `EIGEN` is the directory containing `Eigen/Core`, `CUDA_ARCH` is the NVIDIA compute capability without a decimal point (for example, `80` for SM 8.0), and `PLAT` determines the library file suffix.

4.1 CPU executable

For the default MKL/Pardiso configuration:

```
make MAKEINC=make.inc.local
make -C examples MAKEINC=../make.inc.local
```

This produces the library `libfasteig_<PLAT>.a` at the repository root and the executable `examples/runfasteig_cpu`.

To use Eigen's native sparse LDLT instead of MKL/Pardiso, remove MKL flags from the build consistently:

```
make -C src MAKEINC=../make.inc.local clean
make -C examples MAKEINC=../make.inc.local clean
make MAKEINC=make.inc.local USE_MKL=""
make -C examples MAKEINC=../make.inc.local USE_MKL="" MKL=""
```

4.2 GPU executable

After setting `CUDA_PATH`, `CUDA_ARCH`, and `NVCC` in the local make include, build with:

```
make MAKEINC=make.inc.local USE_CUDA=1
make -C examples MAKEINC=../make.inc.local USE_CUDA=1
```

This creates both `runfasteig_cpu` and `runfasteig_gpu`. The GPU driver selects a CUDA executor for `solver Lanczos` and `solver BlockLanczos`. Other solver names in the GPU executable continue to use their CPU implementations, so use `runfasteig_cpu` for clarity when selecting them.

The CUDA backend requires a matrix stored in one of FASTEIG's input formats. Matrix-free operators are CPU-only in the current driver. Generalized problems should also be run on the CPU; they are currently supported only by Krylov–Schur, which is not dispatched to the CUDA backend.

4.3 Installing and enabling cuDSS

For GPU `SAI`, install cuDSS into the default checkout-local location:

```
tools/install_cudss.sh
make -C examples MAKEINC=../make.inc.local USE_CUDA=1
```

If direct package downloads are unavailable, download the Linux cuDSS archive from NVIDIA and run:

```
tools/install_cudss.sh --prefix /project/path/cudss \
  --tarball /path/to/cudss-linux-archive.tar.xz
source /project/path/cudss/cudss-env.sh
make -C examples MAKEINC=../make.inc.local USE_CUDA=1 \
  CUDSS_PATH="$CUDSS_PATH" CUDSS_LIBDIR="$CUDSS_LIBDIR"
```

The Makefile enables cuDSS automatically when it finds both `include/cudss.h` and `libcudss.so` (or `libcudss.so.0`). To build CUDA support without cuDSS, pass `USE_CUDSS=0`; ordinary `SR`, `LR`, `SM`, `LM`, and `BE` Lanczos runs do not require cuDSS.

5 Preparing Matrices and an Input File

5.1 Matrix requirements and formats

The command-line driver reads real, square sparse matrices. Eigensolvers in this distribution are intended for symmetric/self-adjoint problems. The recommended interchange format is Matrix Market coordinate format (`.mtx`). A minimal symmetric example is:

```
%%MatrixMarket matrix coordinate real symmetric
3 3 5
1 1 2.0
2 1 -1.0
```

```
2 2 2.0
3 2 -1.0
3 3 2.0
```

Matrix Market indices are one-based. For a file declared `symmetric` or `skew-symmetric`, store one triangle plus the diagonal; the loader keeps one triangle and applies the symmetry during matrix-vector products. A real coordinate file is safest: the current reader expects row, column, and one floating-point value per entry.

The parser also accepts:

- `.rsa`: a Harwell–Boeing/RSA-style text matrix in compressed sparse column form; and
- `.fei`: FASTEIG's binary CSC form, containing three `FASTEIG_INT` header values (rows, columns, nonzeros), followed by the zero-based column-pointer array, row-index array, and `FASTEIG_DOUBLE` values.

The extension `.hdf` is recognized by format detection but is not loaded by the current input parser. Do not use it for command-line runs. For a generalized problem, `amatrix` and `bmatrix` must have the same format and dimension; B should be symmetric positive definite.

5.2 Input-deck syntax

Each non-comment record contains exactly a key and a whitespace-separated value. Keys are case-insensitive. A line whose first token begins with `#` or `%` is a comment. Avoid inline comments and paths containing spaces. Matrix paths are interpreted relative to the directory from which the executable is launched, not relative to the input file itself.

A practical standard-problem input file is:

```
# Matrix and output
amatrix ./Sparse_Matrices/SiO.mtx
output_prefix SiO
output\_format text

# Requested eigenpairs
nev 5
which SR
solver Lanczos

# Iteration controls
maxiter 200
maxdim 20
blksize 5
tol 1e-8
seed 1

# Console output
verbosity 2
print\_every 10
```

For a generalized problem, add a matching matrix:

```
amatrix /data/A.mtx
bmatrix /data/B.mtx
solver KrylovSchur
```

```
which SR
nev 10
maxiter 500
maxdim 40
tol 1e-10
output_prefix generalized
output_format text
```

5.3 Input keys

Key	Default	Meaning
`amatrix`	required	Path to the standard matrix A.
`bmatrix`	none	Optional matrix B for $Ax = \lambda Bx$. Currently CPU Krylov–Schur only.
`solver`	Lanczos	Lanczos, BlockLanczos, BlockKrylovSchur, KrylovSchur, Davidson, LOBPCG, Chebyshev, SPPC, RMM-DIIS, or auto.
`nev`	100	Number of requested eigenpairs; must be positive and no greater than the matrix dimension.
`which`	LM	Target: SR (smallest algebraic), LR (largest algebraic), SM (smallest magnitude), LM (largest magnitude), BE (both ends), or SAI (shift-and-invert). Not every solver accepts every target.
`shift`	0.0	Shift σ for SAI, which applies an operator based on $(A - \sigma I)^{-1}$.
`maxiter`	10	Maximum solver iterations or Krylov steps, depending on the algorithm.
`maxdim`	20	Maximum subspace dimension for applicable solvers; SPPC uses it as its split dimension.
`blksize`	5	Initial block size for block algorithms.
`tol`	10^{-6}	Positive convergence tolerance.
`seed`	unspecified	Nonnegative random seed for reproducible initial vectors.
`logfile`	logfile	Output prefix; `output_prefix` is an alias.
`output_format`	text	Output format. Only `text` is currently implemented.
`verbosity`	1	Console level 1 prints the run header and final eigenpairs; level 2 also enables iteration summaries. Values are clamped to 1 or 2.
`print_every`	0	Progress-printing cadence for solvers that use it; zero disables cadence-based progress.
`verbose`	—	Backward-compatible Boolean key that enables normal console output.

The data structure contains a Chebyshev `degree` setting (default 10) and a Davidson `maxblks`

setting, but the current file parser does not assign those two keys. Consequently, command-line input decks use their compiled defaults unless the C++ driver is changed to set them programmatically.

5.4 Choosing a solver and target

For a first run, use `Lanczos` with `SR` or `LR`. The Lanczos family accepts `SR`, `LR`, `SM`, `LM`, and `BE`; scalar Lanczos also accepts `SAI`. `KrylovSchur` supports standard and generalized problems, but generalized `SAI` is rejected. The block Krylov–Schur implementation does not support generalized problems or `SAI`. Chebyshev filtering is designed for an end of the spectrum; the implementation reverses the operator for `LR`/`LM`.

Use `SAI` to find eigenvalues near `shift`. It incurs a sparse factorization setup cost and needs a stored matrix. The CPU backend reports either `MKL PardisoLDLT` or `Eigen SimplicialLDLT`; the GPU backend reports `NVIDIA cuDSS`.

6 Running on CPUs

The input paths in the supplied `examples/SiO.input` assume that the working directory is `examples`. Run:

```
cd examples
./runfasteig_cpu SiO.input
```

The driver exits with status 0 on success, 1 for a command-line or input error, and 2 when the solver returns a nonzero status. With `output_prefix SiO` and `solver Lanczos`, the text log is named `SiO_Lanczos.log`.

Control CPU threading with the OpenMP and math-library environment. For an MKL build on a 16-core allocation:

```
export OMP_NUM_THREADS=16
export MKL_NUM_THREADS=16
export OMP_PLACES=cores
export OMP_PROC_BIND=spread
export OMP_DYNAMIC=false
export MKL_DYNAMIC=false
./runfasteig_cpu SiO.input
```

On a Slurm system, request one process and assign its cores explicitly, for example:

```
srun -N 1 -n 1 -c 16 --cpu-bind=cores \
  ./runfasteig_cpu SiO.input
```

FASTEIG's example driver is a single-process program; do not launch multiple MPI ranks unless each rank is intentionally running an independent problem.

7 Running on NVIDIA GPUs

Use the same input deck and select the backend with the executable name. Keep `solver Lanczos` (or `solver BlockLanczos`) in the deck:

```
cd examples
./runfasteig_gpu SiO.input
```

For an ordinary Lanczos target such as `SR`, no cuDSS installation is needed. For interior eigenvalues, use both `which SAI` and a shift:

```
solver Lanczos
which SAI
shift 0.25
nev 5
```

That run requires a build with `USE_CUDA=1 USE_CUDSS=1`. If the program reports that CUDA SAI requires cuDSS, rebuild after following the cuDSS steps in Section 4.3.

A typical Slurm GPU launch is:

```
export OMP_NUM_THREADS=16
export MKL_NUM_THREADS=16
export OMP_PLACES=cores
export OMP_PROC_BIND=close
export OMP_DYNAMIC=false
export MKL_DYNAMIC=false
srun -n 1 -c 16 --gpus-per-task=1 --gpu-bind=single:1 \
  --cpu-bind=cores ./runfasteig_gpu SiO.input
```

GPU work is not selected merely by compiling with CUDA: run the `runfasteig_gpu` executable and choose Lanczos or BlockLanczos. The current CUDA executor accepts stored CSR data derived from the input matrix; it does not accept the matrix-free callback path.

8 Verification

The repository includes a Python 3 integration harness and a reference spectrum for `SiO.mtx`. After building `examples/runfasteig_cpu`, run from the repository root:

```
python3 tests/scripts/run_suite.py --matrix SiO
```

To test one solver and one end of the spectrum:

```
python3 tests/scripts/run_case.py \
  --solver Lanczos --which SR --matrix SiO
```

The test harness checks reported residuals and eigenvalues against `tests/reference/SiO.out.121mpi`. It requires only Python 3, not SciPy.

9 Troubleshooting

CMake reports that `Eigen/Core` is missing.

Set `EIGEN_DIR` to the directory immediately above `Eigen`, not to the `Eigen` directory itself. Common choices are `/usr/include/eigen3` and a source checkout such as `/opt/eigen-3.4.0`; verify that `<EIGEN_DIR>/Eigen/Core` exists.

CMake tries to clone OpenBLAS.

Install development packages for both BLAS and LAPACK, provide a discoverable vendor implementation, or configure with MKL/LibSci. Otherwise network access is expected for the fallback.

CMake fails while adding `examples` or `tests`.
Reconfigure with `-DBUILD_EXAMPLES=OFF -DBUILD_TESTS=OFF`. Build the drivers with their Makefile.

The matrix file cannot be opened.
Resolve `amatrix` and `bmatrix` from the launch directory. For example, the supplied relative path works after `cd examples`, not necessarily from the repository root.

A generalized problem is rejected.
Use `solver KrylovSchur` and the CPU executable. Ensure A and B have identical dimensions and file-format extensions.

GPU execution says a stored CSR matrix is required.
Supply `amatrix` in `.mtx`, `.rsa`, or `.fei` format. The current GPU driver cannot use a matrix-free callback.

GPU SAI says cuDSS is unavailable.
Confirm that `CUDSS_PATH/include/cudss.h` and a cuDSS shared library under `CUDSS_LIBDIR` exist, then clean and rebuild with `USE_CUDA=1 USE_CUDSS=1`. Source the generated `cudss-env.sh` before both building and running when using a custom prefix.

Performance is lower than expected.
Use a Release build, match thread counts to the allocated cores, disable dynamic OpenMP/MKL resizing, and choose the correct `CUDA_ARCH` for a GPU build. Avoid oversubscribing a batch allocation.

10 Quick Reference

CPU.
```
make MAKEINC=make.inc.local
make -C examples MAKEINC=../make.inc.local
cd examples
./runfasteig_cpu SiO.input
```

GPU.
```
make MAKEINC=make.inc.local USE_CUDA=1
make -C examples MAKEINC=../make.inc.local USE_CUDA=1
cd examples
./runfasteig_gpu SiO.input
```

Compile this guide. From the repository root:

```
pdflatex -output-directory=docs docs/fasteig_user_guide.tex
pdflatex -output-directory=docs docs/fasteig_user_guide.tex
```